Web Services

Why Services Exist
&
Where SOA Came From

(MP1 - GL)

Ikram Garfatta

Higher Institute for Computer Science and Mathematics of Monastir

ikram.garfatta@gmail.com

January 27, 2026

Course structure (tentative roadmap)

1. **Foundations:** why services exist, SOA history, layered & multi-tier architectures
2. **HTTP in depth:** methods, status codes, headers, caching basics
3. **REST as an architectural style:** constraints, resource modeling, URI design
4. **CRUD + beyond:** pagination/filtering/sorting, PUT vs PATCH, error design
5. **Implementation:** building a RESTful service (framework-agnostic, then one framework)
6. **API documentation:** OpenAPI / Swagger and contract thinking
7. **Testing & tooling:** Postman, collections, environments, basic automated tests
8. **Security essentials:** API keys, JWT, OAuth2 concepts, OWASP API risks
9. **Evolution & operations:** versioning, backward compatibility, rate limiting
10. **Perspectives:** REST vs GraphQL (intro), microservices view, event-driven APIs (intro)

Weeks 1–3: Concepts + HTTP

↓

Weeks 4–6: REST + Design

↓

Weeks 7–9: Build + Document + Test

↓

Weeks 10–11: Secure + Evolve + Outlook

Goal: connect **principles** to **practice** without tool lock-in.

Course structure (tentative roadmap)

1. **Foundations:** why services exist, SOA history, layered & multi-tier architectures
2. **HTTP in depth:** methods, status codes, headers, caching basics
3. **REST as an architectural style:** constraints, resource modeling, URI design
4. **CRUD + beyond:** pagination/filtering/sorting, PUT vs PATCH, error design
5. **Implementation:** building a RESTful service (framework-agnostic, then one framework)
6. **API documentation:** OpenAPI / Swagger and contract thinking
7. **Testing & tooling:** Postman, collections, environments, basic automated tests
8. **Security essentials:** API keys, JWT, OAuth2 concepts, OWASP API risks
9. **Evolution & operations:** versioning, backward compatibility, rate limiting
10. **Perspectives:** REST vs GraphQL (intro), microservices view, event-driven APIs (intro)

Weeks 1–3: Concepts + HTTP

↓

Weeks 4–6: REST + Design

↓

Weeks 7–9: Build + Document + Test

↓

Weeks 10–11: Secure + Evolve + Outlook

Goal: connect **principles** to **practice** without tool lock-in.

This roadmap reflects the current plan.
It may evolve depending on pace, prerequisites, and class interests.

Session roadmap

1. Motivation: software dependencies today
2. What is a service?
3. Service-Oriented Architecture (SOA)
4. Historical background: SOAP, WSDL, UDDI
5. Layered architecture of web systems
6. Course positioning and outlook

Name 3 apps you used this week that *depend on another company's API*.

Name 3 apps you used this week that
depend on another company's API.

- **Example answers:** Authentication services, Payment services, Mapping, Messaging, Analytics services

Software dependencies in modern applications

Name 3 apps you used this week that
depend on another company's API.

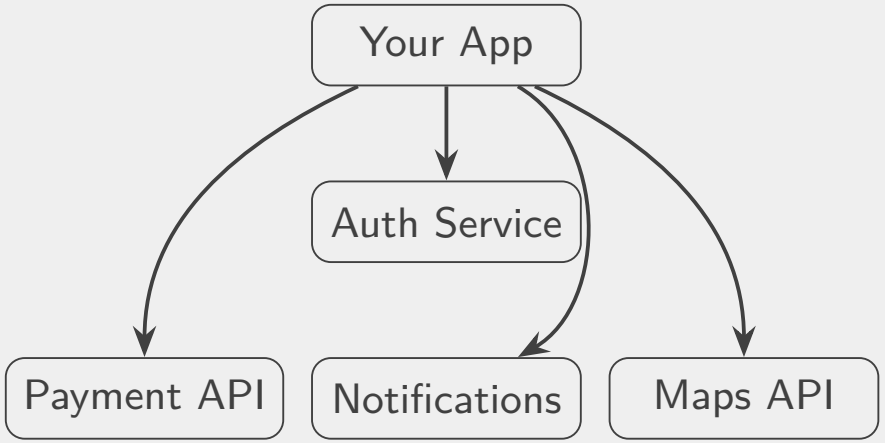
- **Example answers:** Authentication services, Payment services, Mapping, Messaging, Analytics services
- Today: **most systems are systems-of-systems**.

The hidden graph behind one app

- A modern application is rarely standalone:

The hidden graph behind one app

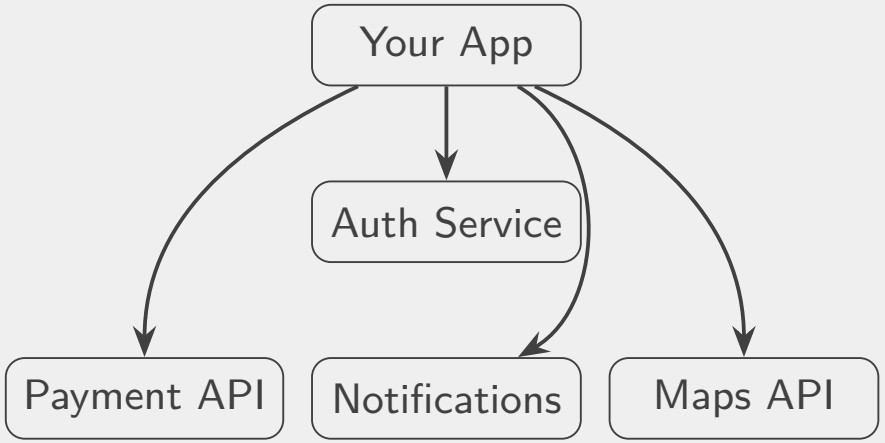
- A modern application is rarely standalone:
  - **Auth** (SSO/OAuth), **payments**, **notifications**, **analytics**,
  - **search**, **recommendations**, **maps**, **cloud storage**.



Same idea inside enterprises:
HR, finance, logistics, CRM

The hidden graph behind one app

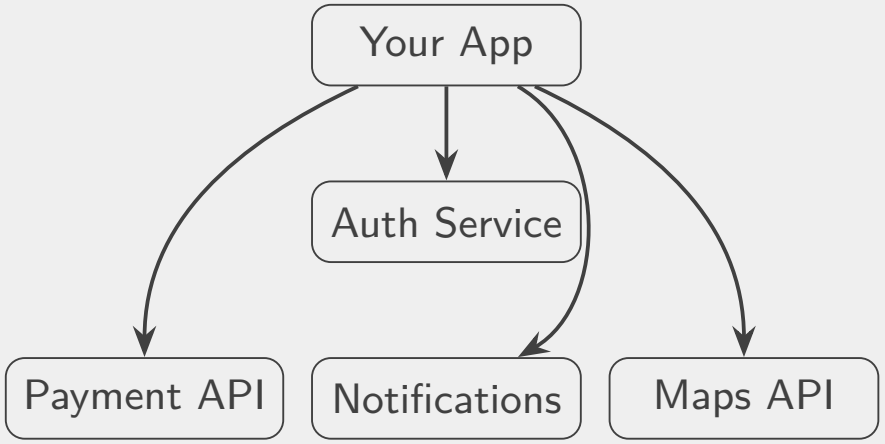
- A modern application is rarely standalone:
  - **Auth** (SSO/OAuth), **payments**, **notifications**, **analytics**,
  - **search**, **recommendations**, **maps**, **cloud storage**.
- These interactions have a name: **services** exposed via **APIs**.



Same idea inside enterprises:
HR, finance, logistics, CRM

The hidden graph behind one app

- A modern application is rarely standalone:
  - **Auth** (SSO/OAuth), **payments**, **notifications**, **analytics**,
  - **search**, **recommendations**, **maps**, **cloud storage**.
- These interactions have a name: **services** exposed via **APIs**.
- **Key promise:** reuse and speed.
- **Key pain:** latency, failures, contract changes.



Same idea inside enterprises:
HR, finance, logistics, CRM

What is a service?

Definition

A **service** is a *software capability* exposed to other systems through a **stable interface**, enabling interaction over a network.

What is a service?

Definition

A **service** is a *software capability* exposed to other systems through a **stable interface**, enabling interaction over a network.

- **Service provider:** offers the capability
- **Service consumer:** uses the capability
- **Contract:** defines how the service is accessed and what it returns

Service vs library vs component

Concept	Description	Coupling
Library	Code reused at build time	Strong
Component	Deployable part of a system	Medium
Service	Remote capability via interface	Designed for loose coupling

Service-Oriented Architecture (SOA)

- **SOA** is an architectural approach based on composing systems from services.
- **Key principles:**
  - Loose coupling
  - Explicit contracts
  - Reuse of capabilities
  - Interoperability

Service-Oriented Architecture (SOA)

- **SOA** is an architectural approach based on composing systems from services.
- **Key principles:**
  - Loose coupling
  - Explicit contracts
  - Reuse of capabilities
  - Interoperability

Modern perspective

In modern systems, many SOA principles are realized through API-based architectures.

Services as organizational boundaries

- In large software systems, different **teams** often own different functionalities.
- Exposing functionality as a **service** helps:
  - define clear responsibilities,
  - enforce explicit **contracts** between teams,
  - allow **independent evolution** of components.
- Services are therefore not only a **technical abstraction**, but also an **organizational design choice**.

Key ideas

- A service interface is a **contract** between teams
- A service interface defines both a **technical boundary** and a **team boundary**.

Bezos API mandate

History of Web-based Systems

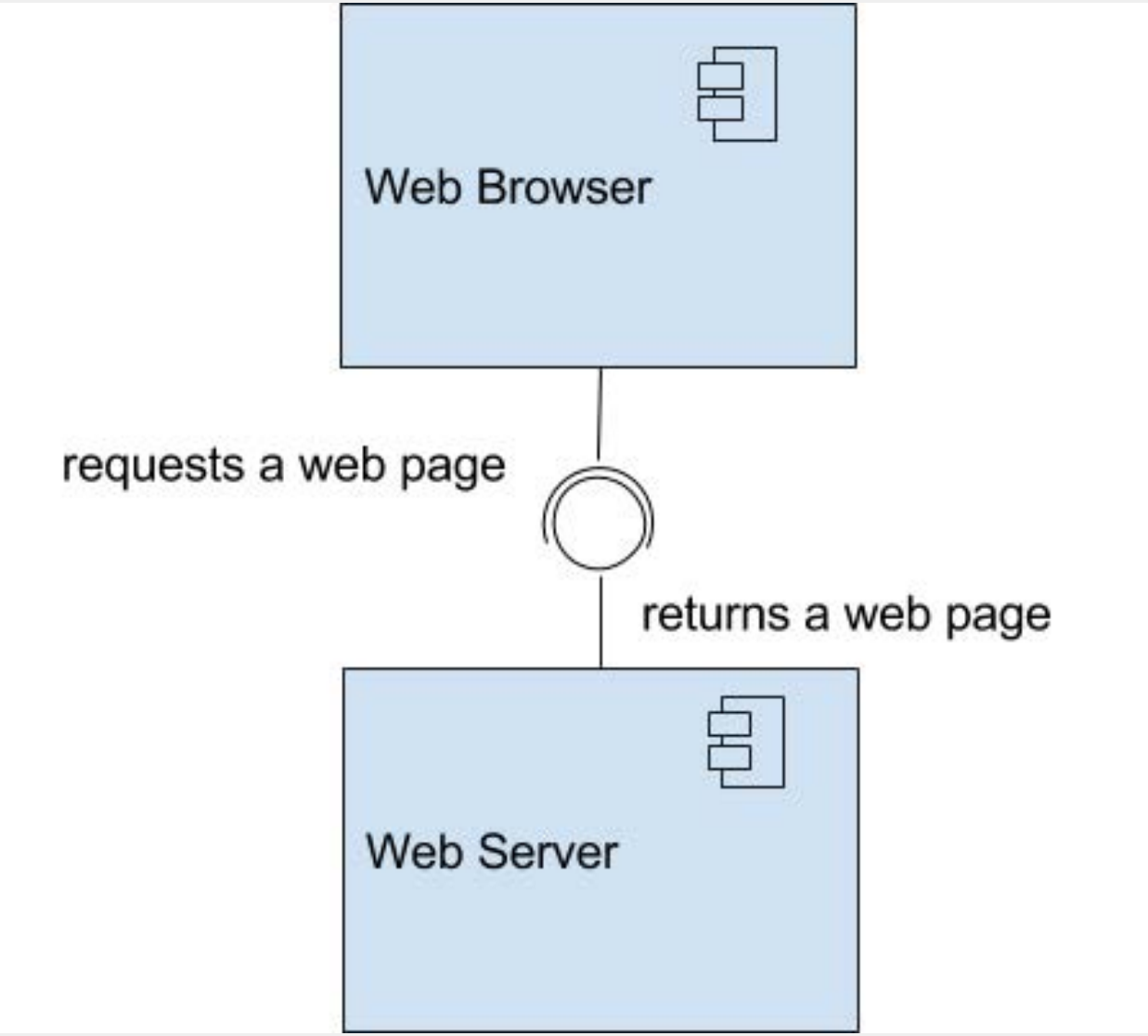
Understanding the Origins:

- **1969**: ARPANET was created, enabling researchers to send small amounts of data between computers.
- **1990**: Tim Berners-Lee introduced the World Wide Web (WWW), inspired by hypertext and association.
- The Internet is the underlying network, while the WWW provides a way to access linked documents and resources.

Impact of the WWW:

- HTML (HyperText Markup Language) enabled structured web pages.
- HTTP (HyperText Transfer Protocol) facilitated communication between browsers and servers.
- Web browsers provided users with access to websites, popularizing the WWW in the early 1990s.

History of Web-based Systems



Static Web Pages

Characteristics:

- Each web page is stored as a separate, fixed HTML file on the server.
- The displayed content is identical to the stored HTML file.
- Modifications require manual updates to each relevant file.

Use Cases and Limitations:

- Ideal for content that changes infrequently, such as personal websites or archival information.
- Simple and cost-effective for small-scale projects.
- Lacks interactivity and customization for users.

Dynamic Web Pages

Characteristics:

- Web pages are generated dynamically in response to user requests.
- Involves interaction with applications, databases, or web services to create content.
- Allows for customization, user interactivity, and scalability.

Advantages:

- Updates are centralized, enabling consistent changes across multiple pages.
- Suitable for complex systems, such as e-commerce sites and social media platforms.
- Facilitates personalized user experiences, like recommendations or dashboards.

Web Applications

Definition and Features:

- A web application is a software application that runs in a web browser and is stored on a remote server.
- Eliminates the need for installation or updates on user devices.
- Platform-independent as long as a compatible browser is available.

Examples and Benefits:

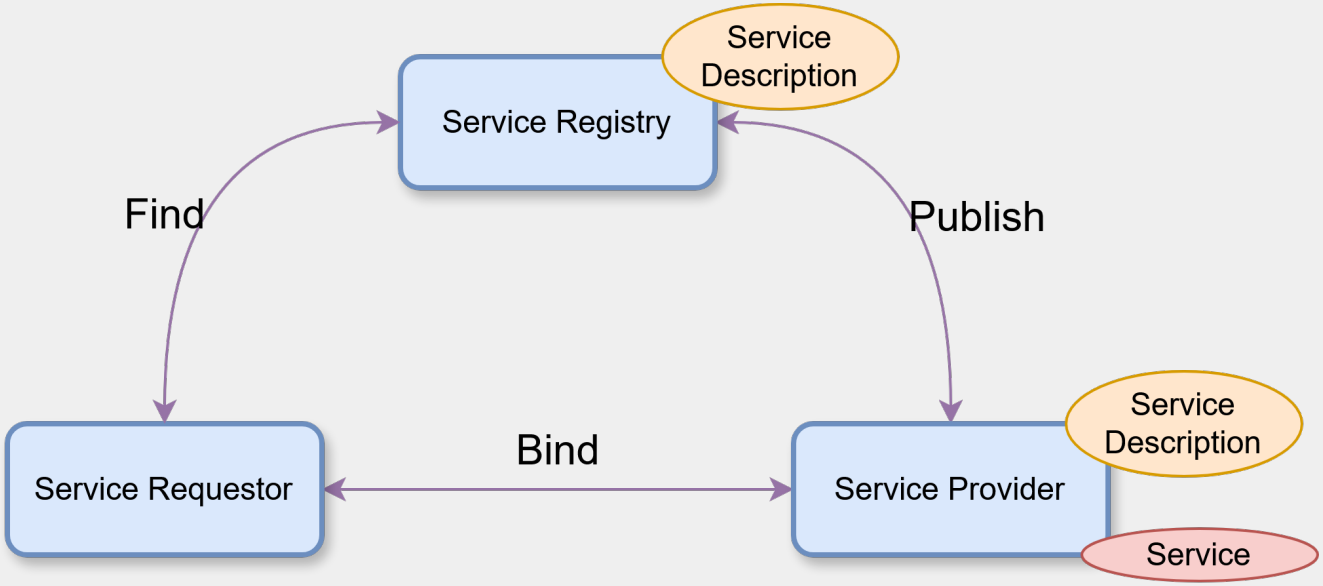
- Examples: Online banking, interactive calendars, e-learning platforms, collaborative tools like Google Docs.
- Provides real-time interactions and seamless updates for users.
- Enhances accessibility across devices and operating systems.

Distributed objects before SOA

- **Technologies:** CORBA, DCOM, Java RMI
- **Goal:** invoke remote methods transparently
- **Limitations:**
  - tight coupling,
  - versioning difficulties,
  - network and firewall constraints.

The classic SOA triangle

- **Provider:** publishes a service
- **Registry:** enables discovery
- **Consumer:** finds and invokes the service



SOAP, WSDL, and UDDI

- **SOAP:** message format and processing rules
- **WSDL:** service interface description
- **UDDI:** service registry and discovery

UDDI

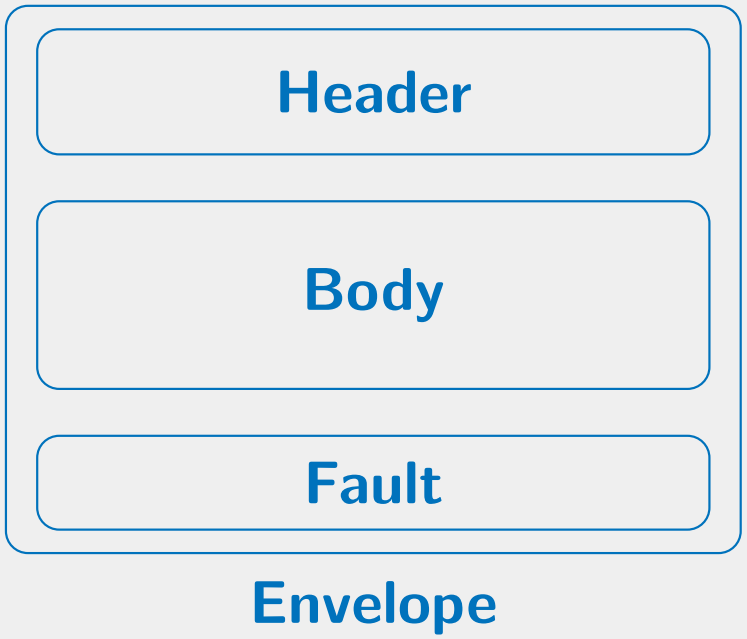
WSDL

SOAP

HTTP

SOAP message structure

- **Envelope:** root element
- **Header:** metadata (optional)
- **Body:** message payload
- **Fault:** error reporting (optional)



Example SOAP message


```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <GetBalance>
      <accountId>123</accountId>
    </GetBalance>
  </soap:Body>
</soap:Envelope>
```

UDDI: service discovery

- Intended as a **registry** for *publishing* and *discovering* services
- Useful concept for understanding service catalogs
- In practice, discovery is often internal and governed

Combining Web Services

Enhancing Application Functionality:

- Web services enable applications to integrate real-time external data and capabilities.
- Examples: Weather APIs for forecasts, financial APIs for stock data, and payment gateways for transactions.

Advantages of Integration:

- Encourages reusability of service components across various applications.
- Supports asynchronous communication, enhancing responsiveness and efficiency.
- Reduces the development effort by leveraging pre-existing services.

Limitations in early SOA implementations

- Complex tooling and specifications
- Heavy reliance on **XML**
- Difficult debugging and performance overhead

These limitations motivated simpler approaches to web services.

REST as a contrasting approach (preview)

- **REST** defined by Roy Fielding (2000)
- Architectural style based on web principles
- Emphasizes stateless interactions and uniform interfaces

Session summary

- **Services** enable integration and independent evolution
- **SOA** introduced enduring architectural principles
- **SOAP-based** web services addressed early interoperability needs
- **REST** will be studied as a modern architectural style